CISC 106 Final Project LLM tracker

in python code, make me a list of Bird dataclasses with the attributes (name: str main_colors: list[str] habitat: str size: str) based on common birds

Here is a list of 20 common North American birds using your Bird dataclass. I've included a variety of colors, habitats, and sizes to make it useful for your bird search engine project.

```
from dataclasses import dataclass


@dataclass
class Bird:
    name: str
    main_colors: list[str]
    habitat: str
    size: str



BIRDS = [
    Bird("American Robin", ["orange", "brown", "black"], "forests", "medium"),
    Bird("Northern Cardinal", ["red", "black"], "woodlands", "medium"),
    Bird("Blue Jay", ["blue", "white", "black"], "forests", "medium"),
    Bird("American Goldfinch", ["yellow", "black", "white"], "fields", "small"),
    Bird("House Sparrow", ["brown", "gray", "black"], "urban", "small"),
    Bird("Mourning Dove", ["gray", "brown", "black"], "woodlands", "medium"),
    Bird("Downy Woodpecker", ["black", "white", "red"], "forests", "small"),
    Bird("Red-bellied Woodpecker", ["red", "black", "white"], "forests", "medium"),
    Bird("Black-capped Chickadee", ["black", "white", "gray"], "woodlands", "small"),
    Bird("Tufted Titmouse", ["gray", "white", "rust"], "forests", "small"),
    Bird("Eastern Bluebird", ["blue", "orange", "white"], "fields", "small"),
```

```
    Bird("House Finch", ["red", "brown", "gray"], "urban", "small"),
    Bird("Common Grackle", ["black", "purple", "green"], "wetlands", "medium"),
    Bird("Mallard", ["green", "brown", "white", "gray"], "wetlands", "large"),
    Bird("Canada Goose", ["brown", "black", "white"], "wetlands", "large"),
    Bird("Great Blue Heron", ["blue", "gray", "white"], "wetlands", "large"),
    Bird("Red-tailed Hawk", ["brown", "white", "red"], "grasslands", "large"),
    Bird("Bald Eagle", ["brown", "white", "yellow"], "forests", "large"),
    Bird("Wild Turkey", ["brown", "black", "red"], "woodlands", "large"),
    Bird("Barn Swallow", ["blue", "orange", "white"], "fields", "small")
]
```

**Suggested categories for your search engine**

To make filtering easier, I'd recommend keeping your attributes consistent:

**Colors:**

- red
- orange
- yellow
- green
- blue
- brown
- black
- white
- gray
- purple

**Habitats:**

- forests
- woodlands

- wetlands
- fields
- grasslands
- urban

**Sizes:**

- small
- medium
- large

**One suggestion:** Since birds can live in multiple habitats, you might eventually consider changing habitat: str to habitat: list[str]. This would let someone find a bird even if it lives in both forests and woodlands.

remake the list with the colors limited to :Blue Red Yellow Green Orange Black White Brown Gray

Here is the revised list of 20 common North American birds using only your nine approved colors:

```
BIRDS = [
    Bird("American Robin", ["Orange", "Brown", "Gray"], "forests", "medium"),
    Bird("Northern Cardinal", ["Red", "Black"], "woodlands", "medium"),
    Bird("Blue Jay", ["Blue", "White", "Black"], "forests", "medium"),
    Bird("American Goldfinch", ["Yellow", "Black", "White"], "fields", "small"),
    Bird("House Sparrow", ["Brown", "Gray", "Black"], "urban", "small"),
    Bird("Mourning Dove", ["Gray", "Brown"], "woodlands", "medium"),
    Bird("Downy Woodpecker", ["Black", "White", "Red"], "forests", "small"),
    Bird("Red-bellied Woodpecker", ["Red", "Black", "White", "Gray"], "forests", "medium"),
    Bird("Black-capped Chickadee", ["Black", "White", "Gray"], "woodlands", "small"),
    Bird("Tufted Titmouse", ["Gray", "White", "Brown"], "forests", "small"),
    Bird("Eastern Bluebird", ["Blue", "Orange", "White"], "fields", "small"),
```

```
    Bird("House Finch", ["Red", "Brown", "Gray"], "urban", "small"),
    Bird("Common Grackle", ["Black", "Blue", "Green"], "wetlands", "medium"),
    Bird("Mallard", ["Green", "Brown", "White", "Gray"], "wetlands", "large"),
    Bird("Canada Goose", ["Brown", "Black", "White"], "wetlands", "large"),
    Bird("Great Blue Heron", ["Blue", "Gray", "White"], "wetlands", "large"),
    Bird("Red-tailed Hawk", ["Brown", "White", "Red"], "grasslands", "large"),
    Bird("Bald Eagle", ["Brown", "White", "Yellow"], "forests", "large"),
    Bird("Wild Turkey", ["Brown", "Black", "Gray"], "woodlands", "large"),
    Bird("Barn Swallow", ["Blue", "Orange", "White"], "fields", "small")
]
```

All colors now match your approved options exactly:

- Blue
- Red
- Yellow
- Green
- Orange
- Black
- White
- Brown
- Gray

This will also make your bird search engine easier to program because you can compare the user's selected colors directly against main_colors.

if im making a drafter program that is like a search engine for birds what fields should mt state dataclass have

For your **Bird Search Engine** in Drafter, I would recommend making your State dataclass store the user's search selections, the birds that match those selections, and information about which page they are currently viewing.

Since your goal is to help someone identify a bird they've seen, you'll want the state to preserve their previous choices when they refine their search or go back to an earlier page.

**Recommended State dataclass**

```
from dataclasses import dataclass


@dataclass
class State:
    selected_colors: list[str]
    selected_habitats: list[str]
    selected_size: str
    matching_birds: list[Bird]
```

**What each field does**

| Field | Purpose | Example |
|---|---|---|
| selected_colors | Stores colors the user selected | ["Blue", "White"] |
| selected_habitats | Stores habitats the user selected | ["forests", "woodlands"] |
| selected_size | Stores the selected bird size | "medium" |
| matching_birds | Stores birds that match the search | [Blue Jay, ...] |

**Initial state**

When the user first opens your program, you could initialize the state like this:

```
State(
    selected_colors=[],
    selected_habitats=[],
    selected_size="",
    matching_birds=BIRDS
```

)

This starts with no filters selected and all birds available.

**How the state works with your routes**

Based on your planned routes, here's how I would use the fields:

| **Route** | **What it changes in State** |
|---|---|
| index | Displays the starting page |
| after_colors | Updates selected_colors |
| after_habitat | Updates selected_habitats |
| after_size | Updates selected_size and filters birds |
| return_to_start | Clears all selections and restores all birds |
| refine_search | Keeps existing selections so the user can modify them |

**A few recommendations**

1. **Keep** BIRDS **outside of State.** Your bird database doesn't need to be stored in State because it isn't changing.
2. **Keep** matching_birds **in State.** This makes it easier to display search results on different pages.
3. **Don't store the current page in State.** Drafter routes already determine which page is displayed.
4. **Use empty lists for unselected filters.** This makes it easier to check whether a user has selected anything.

One important design decision: I would make your search filters cumulative. For example, if someone selects blue and then forests, the results should contain only birds that are both blue and found in forests.

For a bird-search dataclass, tracing multi-agent state workflows can help.

i need 7 routes

For your bird search engine, I would use **7 routes** that let users search by bird characteristics, view results, and refine their search without losing previous selections.

**Recommended 7 routes**

| Route | Purpose |
|---|---|
| index | Starting page; introduces the bird search and lets users begin. |
| after_colors | Processes the colors selected by the user. |
| after_habitat | Processes the habitats selected by the user. |
| after_size | Processes the size selected by the user. |
| search_results | Displays all birds matching the selected characteristics. |
| refine_search | Returns to the selection pages while preserving previous selections. |
| return_to_start | Clears all selections and returns to the starting page. |

**How the routes connect**

**1. index**

Start searching

**2. after_colors**

Select bird colors

**3. after_habitat**

Select habitat

**4. after_size**

Select bird size

**5. search_results**

Display matching birds

**6. refine_search**

Keep previous selections

**7. return_to_start**

Clear everything

**State dataclass**

I would use:

@dataclass

class State:

selected_colors: list[str]

selected_habitats: list[str]

selected_size: str

matching_birds: list[Bird]

**One additional idea:** On your results page, include the bird's name, colors, habitat, and size. You could also let users click a bird to see more information about it.

This gives you a realistic seven-route project without making the program unnecessarily complicated.

This might also interest you.

do these routes return atleast 5 different pages

Yes! Your 7 routes can return at least **5 different types of pages**. However, routes and pages aren't necessarily the same thing. Multiple routes can return the same page layout with different content.

For your project, I would organize them like this:

| **Route** | **Page returned** | **Different page?** |
|---|---|---|
| index | Welcome/start page | Yes |

| | | |
|---|---|---|
| after_colors | Habitat selection page | Yes |
| after_habitat | Size selection page | Yes |
| after_size | Search results page | Yes |
| search_results | Detailed bird results page | Yes |
| refine_search | Search selection page with previous choices saved | Reuses a page |
| return_to_start | Welcome/start page with cleared state | Reuses a page |

**You would have 5 distinct page types and 7 routes.**

One thing to consider: If your instructor requires five different page designs rather than five different page contents, this setup meets that goal with five distinct layouts.

I would keep this structure because it makes sense for a bird identification search engine and gives you enough variety for a Drafter project.

This might also interest you.

what should i import from drafter

For your bird search engine project, I would start with:

from drafter import *

from dataclasses import dataclass

The from drafter import * imports the components and functions you'll likely need, including:

- start_server – runs your Drafter application.
- Page – defines what a page displays.
- Header – creates headings.
- Text – displays text.
- Button – creates buttons that connect to routes.
- SelectBox – lets users choose from options.

- CheckBox – lets users select multiple options.

You'll also need your Bird and State dataclasses.

**My recommendation:** For a CS1 project, importing everything with from drafter import * is perfectly reasonable if that's how your class textbook teaches Drafter.

help me write the index route

For your bird search engine, the index route should be a welcoming starting page that explains the program and lets the user begin identifying a bird.

I'd keep it simple:

```
@route
def index(state: State) -> Page:
    return Page(state, [
        Header("Bird Search Engine", 1),
        Text("Welcome! Use this search engine to help identify a bird you have seen."),
        Text("You can search for birds based on their colors, habitat, and size."),
        Button("Start Searching", "/after_colors")
    ])
```

**What the page looks like**

**Bird Search Engine**

Welcome! Use this search engine to help identify a bird you have seen.

You can search for birds based on their colors, habitat, and size.

**Start Searching**

**How it works**

- @route tells Drafter this is a route.
- index(state: State) receives the current state.
- Page(state, [...]) returns the page and preserves the state.
- Button("Start Searching", "/after_colors") sends the user to your next route.

You can add bird pictures or more information later, but this is a good starting point.

For deploying and scaling a Draf ter-style app, here’s a Cloudflare option.

help me write after colors

Worked for 17s

For your after_colors route, you want the user to select one or more colors they noticed on the bird.

Since your main_colors attribute is a list[str], I recommend allowing users to select **multiple colors** rather than just one.

You can use a CheckBox for each color.

Here is how I would write it using Drafter's RelatedCheckBox component:

```
@route
def after_colors(state: State) -> Page:
    return Page(state, [
        Header("What colors was the bird?", 2),
        Text("Select all the colors you noticed."),

        RelatedCheckBox("colors", "Blue"),
        RelatedCheckBox("colors", "Red"),
        RelatedCheckBox("colors", "Yellow"),
        RelatedCheckBox("colors", "Green"),
        RelatedCheckBox("colors", "Orange"),
        RelatedCheckBox("colors", "Black"),
        RelatedCheckBox("colors", "White"),
        RelatedCheckBox("colors", "Brown"),
        RelatedCheckBox("colors", "Gray"),

        Button("Continue", "after_habitat")
    ])
```

**Why use** RelatedCheckBox**?**

Unlike a regular CheckBox, RelatedCheckBox lets you group multiple choices under one name and receive the selected options as a list.

For example, if the user selects Blue, White, and Black, Drafter sends:

["Blue", "White", "Black"]

**How the information gets saved**

Your next route, after_habitat, will receive the selected colors:

```
@route
def after_habitat(state: State, colors: list[str]) -> Page:
  state.selected_colors = colors


  # Return the habitat selection page
```

This is important: the input name "colors" must match the parameter name colors in the receiving route.

You can read more in the Drafter Forms and Input documentation.

**One recommendation:** Let users select multiple colors because birds often have several noticeable colors. This will make your search engine much more useful!

will i use relatedcheckbox in the next route too

Yes! You can use RelatedCheckBox in your after_habitat route if you want users to select multiple habitats.

For example:

```
@route
def after_habitat(state: State) -> Page:
  return Page(state, [
    Header("Where did you see the bird?", 2),
    Text("Select all habitats that apply."),
```

```
    RelatedCheckBox("habitats", "Forests"),
    RelatedCheckBox("habitats", "Woodlands"),
    RelatedCheckBox("habitats", "Wetlands"),
    RelatedCheckBox("habitats", "Fields"),
    RelatedCheckBox("habitats", "Grasslands"),
    RelatedCheckBox("habitats", "Urban"),


    Button("Continue", "after_size")
  ])
```

**The difference for your next route:**

- after_colors: saves the selected colors.
- after_habitat: saves the selected habitats.
- after_size: could use a regular SelectBox or RadioButton because the user only needs to select one size.

This keeps your search process consistent and easy to use!

does this mean i would need to change my original state to have a list for the habitat attribute

Yes! Since you're allowing users to select multiple habitats, your State dataclass should have selected_habitats: list[str] instead of selected_habitat: str.

I would structure your State like this:

```
@dataclass
class State:
  selected_colors: list[str]
  selected_habitats: list[str]
  selected_size: str
  matching_birds: list[Bird]
```

You **do not** need to change your Bird dataclass unless you also want each bird to have multiple habitats.

However, I would actually recommend changing your Bird dataclass to:

```
@dataclass
class Bird:
    name: str
    main_colors: list[str]
    habitat: list[str]
    size: str
```

This way, a bird like the American Robin can have multiple habitats:

```
Bird("American Robin", ["Orange", "Brown", "Gray"],
     ["forests", "woodlands", "urban"], "medium")
```

This makes your search engine more accurate because birds commonly live in more than one habitat.

update my list of birds to have multiple habitats

Here is your updated list of 20 common North American birds with **multiple habitats** for each bird. This assumes you've changed habitat: str to habitat: list[str] in your Bird dataclass.

```
BIRDS = [
    Bird("American Robin", ["Orange", "Brown", "Gray"],
        ["forests", "woodlands", "urban", "fields"], "medium"),

    Bird("Northern Cardinal", ["Red", "Black"],
        ["woodlands", "forests", "urban"], "medium"),

    Bird("Blue Jay", ["Blue", "White", "Black"],
        ["forests", "woodlands", "urban"], "medium"),
```

```
Bird("American Goldfinch", ["Yellow", "Black", "White"],
    ["fields", "grasslands", "urban"], "small"),

Bird("House Sparrow", ["Brown", "Gray", "Black"],
    ["urban", "fields", "grasslands"], "small"),

Bird("Mourning Dove", ["Gray", "Brown"],
    ["woodlands", "fields", "urban", "grasslands"], "medium"),

Bird("Downy Woodpecker", ["Black", "White", "Red"],
    ["forests", "woodlands", "urban"], "small"),

Bird("Red-bellied Woodpecker", ["Red", "Black", "White", "Gray"],
    ["forests", "woodlands", "urban"], "medium"),

Bird("Black-capped Chickadee", ["Black", "White", "Gray"],
    ["forests", "woodlands"], "small"),

Bird("Tufted Titmouse", ["Gray", "White", "Brown"],
    ["forests", "woodlands", "urban"], "small"),

Bird("Eastern Bluebird", ["Blue", "Orange", "White"],
    ["fields", "grasslands", "woodlands"], "small"),

Bird("House Finch", ["Red", "Brown", "Gray"],
```

```
    ["urban", "fields", "woodlands"], "small"),

  Bird("Common Grackle", ["Black", "Blue", "Green"],
    ["wetlands", "fields", "urban", "grasslands"], "medium"),

  Bird("Mallard", ["Green", "Brown", "White", "Gray"],
    ["wetlands", "fields", "urban"], "large"),

  Bird("Canada Goose", ["Brown", "Black", "White"],
    ["wetlands", "fields", "grasslands", "urban"], "large"),

  Bird("Great Blue Heron", ["Blue", "Gray", "White"],
    ["wetlands", "fields"], "large"),

  Bird("Red-tailed Hawk", ["Brown", "White", "Red"],
    ["forests", "grasslands", "fields", "woodlands"], "large"),

  Bird("Bald Eagle", ["Brown", "White", "Yellow"],
    ["forests", "wetlands"], "large"),

  Bird("Wild Turkey", ["Brown", "Black", "Gray"],
    ["woodlands", "forests", "fields"], "large"),

  Bird("Barn Swallow", ["Blue", "Orange", "White"],
    ["fields", "grasslands", "wetlands", "urban"], "small")
]
```

**Your habitat options**

Make sure these match the names used in your after_habitat route:

- Forests
- Woodlands
- Wetlands
- Fields
- Grasslands
- Urban

**Important:** Python strings are case-sensitive. I'd recommend using the same capitalization everywhere. For example, use "Forests" instead of "forests" in both your bird database and your checkboxes.

That way, your search comparisons will work correctly!

This might also interest you.

help me write search_results

Worked for 12s

Your search_results route is where the actual bird identification happens! It should take the user's selected colors, habitats, and size and compare them against your BIRDS list.

I would break it into three steps:

1. Save the selected size.
2. Find birds that match the user's selections.
3. Display the matching birds.

**1. Search results route**

Here is how I would write it using your current Bird and State dataclasses:

```
@route
def search_results(state: State, size: str) -> Page:
    state.selected_size = size
```

```
  state.matching_birds = []

  for bird in BIRDS:
    color_match = (not state.selected_colors or
                   any(color in bird.main_colors for color in state.selected_colors))

    habitat_match = (not state.selected_habitats or
                     any(habitat in bird.habitat for habitat in state.selected_habitats))

    size_match = (not state.selected_size or
                  bird.size == state.selected_size)

    if color_match and habitat_match and size_match:
      state.matching_birds.append(bird)

  content = [
    Header("Your Bird Search Results", 1),
    Text("Here are the birds that match your selections:")
  ]

  if state.matching_birds == []:
    content.append(Text("Sorry! No birds matched your selections. Try refining your
search."))
  else:
    for bird in state.matching_birds:
      content.append(Header(bird.name, 2))
```

```
        content.append(Text("Colors: " + ", ".join(bird.main_colors)))
        content.append(Text("Habitats: " + ", ".join(bird.habitat)))
        content.append(Text("Size: " + bird.size))

    content.append(Button("Refine Search", "/refine_search"))
    content.append(Button("Start Over", "/return_to_start"))

    return Page(state, content)
```

**2. Understanding the filtering**

The most important part is this section:

| **Variable** | **What it checks** |
|---|---|
| color_match | Does the bird have at least one selected color? |
| habitat_match | Does the bird live in at least one selected habitat? |
| size_match | Does the bird match the selected size? |

The any() function checks whether at least one item in a list matches.

For example, if someone selects:

- Blue
- White
- Forests
- Small

The program will find birds that have at least one of those colors, live in forests, and are small.

**3. One important detail**

Your State dataclass should look like this:

```
@dataclass
class State:
    selected_colors: list[str]
    selected_habitats: list[str]
    selected_size: str
    matching_birds: list[Bird]
```

Also, make sure your earlier routes save the selections into State before navigating to the results page.

**My recommendation:** Keep the any() logic for colors and habitats. Someone who selects blue AND white probably wants to see birds that have either color, rather than only birds that have both.

Next, I can help you write refine_search so users can change their selections without losing their previous choices.

Review my bird search routes tomorrow

Tap to set up

This might also interest you.

can you do this without the any function

Yes! You can replace any() with regular for loops and Boolean variables. This is also a good way to practice loops for your CS1 project.

Here is your revised search_results route:

```
@route
def search_results(state: State, size: str) -> Page:
    state.selected_size = size
    state.matching_birds = []

    for bird in BIRDS:
        color_match = False
```

```
    habitat_match = False

    for color in state.selected_colors:
        if color in bird.main_colors:
            color_match = True

    for habitat in state.selected_habitats:
        if habitat in bird.habitat:
            habitat_match = True

    if state.selected_colors == []:
        color_match = True

    if state.selected_habitats == []:
        habitat_match = True

    size_match = (state.selected_size == "" or
                  bird.size == state.selected_size)

    if color_match and habitat_match and size_match:
        state.matching_birds.append(bird)

content = [
    Header("Your Bird Search Results", 1),
    Text("Here are the birds that match your selections:")
]
```

```
  if state.matching_birds == []:
    content.append(Text("Sorry! No birds matched your selections."))
  else:
    for bird in state.matching_birds:
      content.append(Header(bird.name, 2))
      content.append(Text("Colors: " + ", ".join(bird.main_colors)))
      content.append(Text("Habitats: " + ", ".join(bird.habitat)))
      content.append(Text("Size: " + bird.size))

  content.append(Button("Refine Search", "/refine_search"))
  content.append(Button("Start Over", "/return_to_start"))

  return Page(state, content)
```

**How the replacement works**

Instead of using any(), we start with:

color_match = False

Then we loop through the selected colors:

```
for color in state.selected_colors:
  if color in bird.main_colors:
    color_match = True
```

If even one selected color matches, color_match becomes True.

We do the same thing for habitats.

**Why this is useful:** You're practicing for loops, Boolean variables, and conditional statements instead of relying on a built-in function. This is perfectly reasonable for a CS1 project.

what does the join function do

The .join() function combines all the items in a list into one string, with a separator of your choice between each item.

In your search_results route, you have:

", ".join(bird.main_colors)

Let's say a bird has these colors:

bird.main_colors = ["Blue", "White", "Black"]

The .join() function turns that list into:

"Blue, White, Black"

**Breaking it down**

The syntax is:

"separator".join(list)

- ", " is the separator (a comma followed by a space).
- bird.main_colors is the list being combined.

**More examples**

| **Code** | **Result** |
|---|---|
| ", ".join(["Blue", "Red"]) | "Blue, Red" |
| " ".join(["Blue", "Red"]) | "Blue Red" |
| " - ".join(["Blue", "Red"]) | "Blue - Red" |
| "/".join(["Blue", "Red"]) | "Blue/Red" |

**Why use it in your bird search engine?**

Your bird.main_colors and bird.habitat attributes are lists. However, Text() needs text to display.

So:

Text("Colors: " + ", ".join(bird.main_colors))

displays:

**Colors: Blue, White, Black**

Without .join(), Python wouldn't let you concatenate a string and a list directly.

It's a useful function for displaying lists in a readable format!

what would happen if i tried to run my program right now

Based on the code we've written so far, your program would likely have some errors or incomplete functionality if you tried to run it right now.

Here's what I would expect:

**1. What should work**

- Your Bird dataclass and BIRDS list should load.
- Your State dataclass should initialize.
- Your index route should display the welcome page.
- Your after_colors route should display the color selection page.

**2. Potential problems**

| **Problem** | **Why it might happen** |
|---|---|
| RelatedCheckBox | The selected colors and habitats may not be passed correctly if the route parameters aren't configured properly. |
| after_habitat | Needs to receive and save the selected colors. |
| after_size | Needs to receive and save the selected habitats. |
| search_results | Depends on all three selections being passed correctly. |
| refine_search | May not work yet if you haven't implemented it. |
| return_to_start | May not work yet if you haven't implemented it. |

**3. Your biggest potential issue**

I would double-check how you're using RelatedCheckBox and SelectBox.

For example, your route:

@route

def after_size(state: State, habitats: list[str]) -> Page:

expects Drafter to pass a parameter called habitats.

That means the previous page must submit its selections using the matching input name.

Also, your State needs an initial value:

```
initial_state = State(
  selected_colors=[],
  selected_habitats=[],
  selected_size="",
  matching_birds=BIRDS
)
```

**4. What I would do next**

I recommend testing your program in this order:

1. Run the program and see if the welcome page loads.
2. Click Start Searching.
3. Select two colors and continue.
4. Select two habitats and continue.
5. Select a size and continue.
6. Check whether the results match your selections.

**One thing I'd like to see:** Paste your entire current code, including your imports, dataclasses, bird list, and routes.

I can go through it line by line and identify exactly what will work, what will cause errors, and what needs fixing before you run it.

help me write refune_search

For your refine_search route, you want users to be able to go back and change their bird characteristics **without losing their previous selections**.

I recommend having this route return users to the color selection page, but keep their existing selections in State.

That way, they can change colors, habitats, or size before searching again.

**1. Write the route**

```
@route
def refine_search(state: State) -> Page:
  return Page(state, [
    Header("Refine Your Search", 1),
    Text("What would you like to change?"),

    Button("Change Colors", "/after_colors"),
    Button("Change Habitat", "/after_habitat"),
    Button("Change Size", "/after_size"),

    Button("View Current Results", "/search_results"),
    Button("Start Over", "/return_to_start")
  ])
```

**2. How it works**

- **Change Colors:** Returns to your color selection page.
- **Change Habitat:** Returns to your habitat selection page.
- **Change Size:** Returns to your size selection page.
- **View Current Results:** Lets the user see their existing results.
- **Start Over:** Clears their selections.

**3. Important consideration**

Your State should preserve the existing selections when the user enters this route:

@dataclass

class State:

selected_colors: list[str]

selected_habitats: list[str]

selected_size: str

matching_birds: list[Bird]

Returning Page(state, [...]) preserves the state, but your selection pages will also need to be configured to display the previously selected values if you want users to see their old choices.

Also, your search_results route currently requires a size parameter, so the "View Current Results" button will need to be adjusted to work with the saved state rather than requiring a new size input.

**My recommendation:** Keep this route as a separate menu page. It gives users more control over refining their search instead of forcing them to go through every selection again.

Review my Drafter bird search project tomorrow

Tap to set up

does the code you gave fit with ur recomendation

Mostly yes! The refine_search code follows my recommendation, but there are **two things that need adjusting** to make it work properly with your existing routes.

**1. The selection pages need to preserve previous choices**

When someone clicks "Change Colors," they should see their previously selected colors.

Your after_colors route should eventually be able to display those previous selections rather than starting with everything unchecked.

**2. Fix the View Current Results button**

Your current search_results route requires a size parameter:

def search_results(state: State, size: str) -> Page:

But your refine page doesn't send a new size parameter.

Change it to:

@route

def search_results(state: State) -> Page:

Then remove this line:

state.selected_size = size

Your route will use the size already saved in State.

**My recommended setup**

Your refine_search route can stay exactly as written:

@route

def refine_search(state: State) -> Page:

```
  return Page(state, [
    Header("Refine Your Search", 1),
    Text("What would you like to change?"),
    Button("Change Colors", "/after_colors"),
    Button("Change Habitat", "/after_habitat"),
    Button("Change Size", "/after_size"),
    Button("View Current Results", "/search_results"),
    Button("Start Over", "/return_to_start")
  ])
```

Works with your design

The main thing left is making sure your selection routes preserve and update the existing State values correctly.

if i dont update the size in search_results when will it update

You would update the size in your after_size **route**, when the user submits their new selection.

For example, your after_size route should look something like this:

@route

def after_size(state: State, habitats: list[str]) -> Page:

```
  state.selected_habitats = habitats


  return Page(state, [
    Header("What size was the bird?", 2),
    Text("Select from these sizes"),
    SelectBox("size", ["Small", "Medium", "Large"]),
    Button("Continue", "/search_results")
  ])
```

Then your search_results route receives the new size:

```
@route
def search_results(state: State, size: str) -> Page:
  state.selected_size = size


  # Filter birds using the updated state
```

**Here's the important distinction:**

- after_size displays the size choices.
- search_results receives the user's selected size and saves it in State.
- refine_search doesn't need to change the size because it preserves the previously saved value.

So I would actually keep size: str in search_results. My earlier suggestion to remove it was incorrect for your current route design.

You just need to make sure that when users click "View Current Results" from refine_search, they don't have to submit a new size. You could instead make that button lead to a separate route that uses the already-saved selections.

make the refine_search based on this

Based on your current routes, I would make refine_search a menu that lets users choose which characteristic they want to change. It will preserve their existing selections in State.

```
@route
def refine_search(state: State) -> Page:
  return Page(state, [
    Header("Refine Your Search", 1),
    Text("What would you like to change?"),

    Button("Change Colors", "/after_colors"),
    Button("Change Habitat", "/after_habitat"),
    Button("Change Size", "/after_size"),

    Button("Start Over", "/return_to_start")
  ])
```

**How this works with your other routes**

| **User choice** | **What happens** |
|---|---|
| Change Colors | Goes to after_colors |
| Change Habitat | Goes to after_habitat |
| Change Size | Goes to after_size |
| Start Over | Clears all selections |

**One important detail**

Your search_results route should keep:

```
def search_results(state: State, size: str) -> Page:
  state.selected_size = size
```

This means users who change their size submit a new size, while users who change colors or habitats need to keep their previously saved size.

**I recommend not including a "View Current Results" button yet**, because your current search_results route expects a new size input. You can add that functionality later with a separate route that uses the saved selections.

should i write a view current results route

Yes! I recommend adding a view_current_results route. It will make your program easier to use because someone can return to their existing search results without having to select their size again.

This is especially useful when someone goes to refine_search but decides they don't actually want to change anything.

**1. Write the route**

You can reuse most of your existing search_results code:

```
@route
def view_current_results(state: State) -> Page:
  content = [
    Header("Your Bird Search Results", 1),
    Text("Here are your current matching birds:")
  ]

  if state.matching_birds == []:
    content.append(Text("No birds matched your selections."))
  else:
    for bird in state.matching_birds:
      content.append(Header(bird.name, 2))
      content.append(Text("Colors: " + ", ".join(bird.main_colors)))
      content.append(Text("Habitats: " + ", ".join(bird.habitat)))
      content.append(Text("Size: " + bird.size))
```

```
    content.append(Button("Refine Search", "/refine_search"))
    content.append(Button("Start Over", "/return_to_start"))

    return Page(state, content)
```

**2. Update your refine_search route**

Add a button that takes users to this new route:

```
@route
def refine_search(state: State) -> Page:
    return Page(state, [
        Header("Refine Your Search", 1),
        Text("What would you like to change?"),

        Button("Change Colors", "/after_colors"),
        Button("Change Habitat", "/after_habitat"),
        Button("Change Size", "/after_size"),

        Button("View Current Results", "/view_current_results"),
        Button("Start Over", "/return_to_start")
    ])
```

**3. Your final route list**

You now have **8 routes**:

1. index
2. after_colors
3. after_habitat
4. after_size
5. search_results

6. refine_search
7. view_current_results
8. return_to_start

**Important:** search_results calculates and updates the matching birds, while view_current_results simply displays the already-calculated results. This prevents users from having to repeat their search.

would it be okay to run my code now>

Yes! I would actually recommend running it now rather than waiting until everything is finished. You'll catch errors much earlier.

However, before running it, check these things:

**Pre-run checklist**

- Your Bird dataclass has habitat: list[str].
- Your State dataclass has all four fields.
- Your BIRDS list uses consistent capitalization for colors and habitats.
- Your index route works.
- Your after_colors route saves selected colors.
- Your after_habitat route saves selected habitats.
- Your after_size route saves the selected size.
- Your search_results route filters and displays matching birds.
- Your refine_search route preserves previous selections.
- Your view_current_results route displays saved results.
- Your return_to_start route clears the selections.

**What I would test first**

Try this sequence:

1. Open the program.
2. Select Blue and White.
3. Select Forests.

4. Select Small.
5. Check whether the results include the expected birds.
6. Click Refine Search.
7. Change your size to Medium.
8. Check whether the results update.

My recommendation

**Run it now!**

One thing to watch for: Drafter's RelatedCheckBox and SelectBox input names and route parameters must match exactly.

If you get an error, send me the error message and your code. We can debug it one step at a time.

when i select colors it doesnt have the names just boxes to click

That sounds like the issue is with how your RelatedCheckBox components are written. You need to make sure each checkbox has a label that displays the color next to it.

Try writing your after_colors route like this:

```
@route
def after_colors(state: State) -> Page:
  return Page(state, [
    Header("What colors was the bird?", 2),
    Text("Select all the colors you noticed."),

    RelatedCheckBox("colors", "Blue"),
    RelatedCheckBox("colors", "Red"),
    RelatedCheckBox("colors", "Yellow"),
    RelatedCheckBox("colors", "Green"),
    RelatedCheckBox("colors", "Orange"),
    RelatedCheckBox("colors", "Black"),
```

```
    RelatedCheckBox("colors", "White"),
    RelatedCheckBox("colors", "Brown"),
    RelatedCheckBox("colors", "Gray"),

    Button("Continue", "/after_habitat")
  ])
```

If you're still seeing only empty boxes, I want to check one thing:

**Can you paste your current** after_colors **code and tell me whether you're using** RelatedCheckBox **or** CheckBox**?**

The exact component syntax matters here, and I don't want to give you a fix that doesn't match your version of Drafter.

It still isn’t showing the names

I found the issue! Your RelatedCheckBox syntax is actually correct. You're missing the text labels after each checkbox.

In Drafter, RelatedCheckBox creates the clickable box, but it does not automatically display the option's name beside it. You have to add the text separately.

**Corrected code**

Try this:

```
@routedef select_colors(state: State) -> Page:    """    Returns a page with color options.    Uses RelatedCheckBox so the user can select multiple colors.    """    return Page(state, [        Header("What colors was the bird?", 2),        Text("Select all the colors you noticed."),        RelatedCheckBox("colors", "Blue"),        " Blue",        RelatedCheckBox("colors", "Red"),        " Red",        RelatedCheckBox("colors", "Yellow"),        " Yellow",        RelatedCheckBox("colors", "Green"),        " Green",        RelatedCheckBox("colors", "Orange"),        " Orange",        RelatedCheckBox("colors", "Black"),        " Black",        RelatedCheckBox("colors", "White"),        " White",        RelatedCheckBox("colors", "Brown"),        " Brown",        RelatedCheckBox("colors", "Gray"),        " Gray",        Button("Continue", "select_habitat")    ])
```

**Why this works**

For example:

```
RelatedCheckBox("colors", "Blue")," Blue",
```

- RelatedCheckBox("colors", "Blue") creates the checkbox.
- " Blue" displays the word Blue beside it.

This is actually the format used in the official Drafter RelatedCheckBox documentation.

Your original checkbox syntax was correct!

You just needed to add the text after each checkbox. You can use this same approach for your habitat selection page, too.

wont this mean that the state changes before the user input

Good question! And you're right to think about when the State actually changes.

**No, removing the** habitats **parameter from** after_size **does not change the State before user input.** It simply prevents the route from requiring new habitat input.

Here's the distinction:

**How the State changes**

Suppose your user initially selects:

- Colors: Blue, White
- Habitat: Forests
- Size: Small

Your State now stores:

```
State(
  selected_colors=["Blue", "White"],
  selected_habitats=["Forests"],
  selected_size="Small",
  matching_birds=[]
)
```

Now the user clicks **Change Size**.

1. refine_search opens after_size.
2. The existing State is preserved.
3. The user chooses Medium.

4. They click Continue.
5. search_results receives "Medium" and updates state.selected_size.

Your after_size route can therefore look like this:

```
@route
def after_size(state: State) -> Page:
  return Page(state, [
    Header("What size was the bird?", 2),
    Text("Select from these sizes"),
    SelectBox("size", ["Small", "Medium", "Large"]),
    Button("Continue", "/search_results")
  ])
```

And your search_results route handles the update:

```
@route
def search_results(state: State, size: str) -> Page:
  state.selected_size = size

  # Your bird filtering code
```

**The important concept**

Think of it this way:

- **Displaying a selection page** does not change the saved selection.
- **Submitting a selection** changes the saved selection.
- **Navigating between pages** preserves the existing State.

This is exactly what you want for your refine search feature!

but if i change to edit state in the function its changed in wont that cause a problem

Yes, you're thinking about an important Python concept! The issue is that **State is mutable**, meaning that when you change a field inside a route, the change can carry over to other routes.

However, in your particular situation, this isn't a problem as long as you only update State when the user submits new input.

**Here's the difference**

Imagine your user has:

state.selected_size = "Small"

They click "Change Size."

When after_size runs, it should only display the choices. It should NOT change state.selected_size.

```
@route
def after_size(state: State) -> Page:
  return Page(state, [
    Header("What size was the bird?", 2),
    SelectBox("size", ["Small", "Medium", "Large"]),
    Button("Continue", "/search_results")
  ])
```

At this point, their saved size is still "Small".

Only when they submit their new selection does this happen:

```
@route
def search_results(state: State, size: str) -> Page:
  state.selected_size = size
```

Now the size changes to "Medium".

**My recommendation**

Keep your routes separated this way:

| **Route** | **Changes State?** |
|---|---|
| index | No |

| | |
|---|---|
| after_colors | No, just displays choices |
| after_habitat | No, just displays choices |
| after_size | No, just displays choices |
| search_results | Yes, saves submitted selections and calculates results |
| refine_search | No |
| view_current_results | No |
| return_to_start | Yes, clears everything |

**One exception:** Your color and habitat selections also need to be saved somewhere when submitted. You can do that in the routes that receive their form inputs.

This separation makes your program much easier to debug and prevents accidentally erasing someone's previous search!

so what how should i change my code to work like that

You should organize your routes so that **each selection page only displays choices**, and the route that receives the submitted choices is responsible for updating the state.

For your bird search engine, the flow should look like this:

1. select_colors displays color choices.
2. select_habitat receives and saves the selected colors, then displays habitat choices.
3. after_size receives and saves the selected habitats, then displays size choices.
4. search_results receives and saves the selected size, then filters the birds.

Here is how I would change the code.

**1. Color selection**

@route

def select_colors(state: State) -> Page:

```
  return Page(state, [
    Header("What colors was the bird?", 2),
    Text("Select all the colors you noticed."),
    RelatedCheckBox("colors", "Blue"),
    RelatedCheckBox("colors", "Red"),
    RelatedCheckBox("colors", "Yellow"),
    RelatedCheckBox("colors", "Green"),
    RelatedCheckBox("colors", "Orange"),
    RelatedCheckBox("colors", "Black"),
    RelatedCheckBox("colors", "White"),
    RelatedCheckBox("colors", "Brown"),
    RelatedCheckBox("colors", "Gray"),
    Button("Continue", "/select_habitat")
  ])
```

**2. Habitat selection**

This route receives the colors selected on the previous page and saves them.

```
@route
def select_habitat(state: State, colors: list[str]) -> Page:
  state.selected_colors = colors

  return Page(state, [
    Header("Where did you see the bird?", 2),
    Text("Select all habitats that apply."),
    RelatedCheckBox("habitats", "Forests"),
    RelatedCheckBox("habitats", "Woodlands"),
    RelatedCheckBox("habitats", "Wetlands"),
```

```
    RelatedCheckBox("habitats", "Fields"),
    RelatedCheckBox("habitats", "Grasslands"),
    RelatedCheckBox("habitats", "Urban"),
    Button("Continue", "/after_size")
  ])
```

**3. Size selection**

This route receives and saves the habitats, then displays the size options.

```
@route
def after_size(state: State, habitats: list[str]) -> Page:
  state.selected_habitats = habitats

  return Page(state, [
    Header("What size was the bird?", 2),
    Text("Select the size that best describes the bird."),
    SelectBox("size", ["Small", "Medium", "Large"]),
    Button("Continue", "/search_results")
  ])
```

**4. Search results**

This route receives the size and saves it before filtering.

```
@route
def search_results(state: State, size: str) -> Page:
  state.selected_size = size

  # Put your bird-filtering code here.
```

**5. Refining your search**

Your refine_search route should not change any of the saved selections. It should just let the user choose which selection page to revisit.

```
@route
def refine_search(state: State) -> Page:
  return Page(state, [
    Header("Refine Your Search", 1),
    Text("What would you like to change?"),
    Button("Change Colors", "/select_colors"),
    Button("Change Habitat", "/select_habitat"),
    Button("Change Size", "/after_size"),
    Button("View Current Results", "/view_current_results"),
    Button("Start Over", "/return_to_start")
  ])
```

**One important thing:** This version assumes that when a user clicks Continue, Drafter passes the new form selections to the next route. When refining, the selection pages may also need to preserve or prefill previous choices. We can work on that after the basic routes work.

The main idea is that you should only update a selection when the user submits that selection—not when they simply open a page.

but thats how my code is now that gave me the error

You're right—my suggested code repeated the setup that was already causing your error. The problem is likely that **your** after_size **route requires a** habitats **argument, but when you click "Change Size" from** refine_search**, Drafter doesn't have a new** habitats **input to pass to it.**

The fix is to make after_size work in both situations:

- When the user comes from select_habitat, the habitats have already been saved.
- When the user comes from refine_search, the habitats are already in state.selected_habitats.

**Change** after_size **to this:**

```
@route
def after_size(state: State) -> Page:
  return Page(state, [
    Header("What size was the bird?", 2),
    Text("Select the size that best describes the bird."),
    SelectBox("size", ["Small", "Medium", "Large"]),
    Button("Continue", "/search_results")
  ])
```

Notice that I removed both habitats from the parameters and:

state.selected_habitats = habitats

You don't need either because select_habitat should save the habitats before sending the user to after_size.

**Your route flow should now be:**

**select_colors**

Saves selected colors in state

**select_habitat**

Saves selected habitats in state

**after_size**

Displays size choices without changing state

**search_results**

Saves size and filters birds

The refine_search route can also send users directly to after_size because it no longer requires a habitats argument.

**Can you paste your current** select_habitat**, after_size, and** refine_search **functions?** I want to check how your buttons and route parameters connect so we can fix the actual error rather than guess at another solution.

I can see the issue now. Your original route sequence works when someone follows it from beginning to end, but it breaks when someone uses Refine Search.

The problem is specifically this line in your select_size route:

def select_size(state:State, habitats:list[str])->Page:

When someone clicks Change size in refine_search, Drafter goes directly to select_size. It doesn't have a newly submitted habitats value to pass along, so you get the missing-argument error.

You should use the saved state instead. Here's how to fix it without changing the rest of your design.

**1. Change select_size**

Replace your current function with:

@routedef select_size(state:State)->Page: """ displays the size options without changing the saved habitats """ return Page(state,[ Header("What size was the bird?",2), Text("Select from these sizes"), SelectBox("size",["Small","Medium","Large"]), Button("Continue", "/search_results") ])

Notice that we removed:

- habitats:list[str] from the parameters.
- state.selected_habitats = habitats.

You don't need to save habitats here because select_habitat already saved them.

**2. There's one more route with the same problem: select_habitat**

Your refine_search button for changing habitats goes directly to select_habitat, but that function requires colors:

def select_habitat(state:State, colors:list[str])->Page:

So even after fixing select_size, clicking Change habitats will cause a similar error.

The cleanest fix is to separate the routes that save submitted answers from the routes that display selection pages.

**3. Make these changes**

Change the Continue button in select_colors:

Button("Continue", "/after_colors")

Add this route to save the colors and display the habitat page:

@routedef after_colors(state: State, colors: list[str]) -> Page:  state.selected_colors = colors  return select_habitat(state)

Remove the colors parameter from select_habitat:

@routedef select_habitat(state: State) -> Page:  return Page(state, [  Header("What habitat was the bird in?", 2),  Text("Select the habitat"),  RelatedCheckBox("habitats", "Forest"),  " Forest",  RelatedCheckBox("habitats", "Woodlands"),  " Woodlands", RelatedCheckBox("habitats", "Wetlands"),  " Wetlands",  RelatedCheckBox("habitats", "Fields"),  " Fields",  RelatedCheckBox("habitats", "Grasslands"),  " Grasslands", RelatedCheckBox("habitats", "Urban"),  " Urban",  Button("Continue", "/after_habitat") ])

Add this route to save the habitats and display the size page:

@routedef after_habitat(state: State, habitats: list[str]) -> Page:  state.selected_habitats = habitats  return select_size(state)

Your select_size route should be the parameterless version from above.

**4. Update your refine_search buttons**

Now all three buttons can go directly to their selection pages:

Button("Change colors", "/select_colors"),Button("Change habitats", "/select_habitat"),Button("Change size", "/select_size"),

**How the routes work now**

select_colors

after_colors

Saves colors

select_habitat

after_habitat

Saves habitats

select_size

search_results

The important distinction is that select_colors, select_habitat, and select_size only display forms. The after_colors and after_habitat routes save the submitted answers.

This lets users revisit any selection page without needing to submit answers from the pages before it.

do i need to save the selected size as well

Yes! You are already doing that correctly in your search_results route:

@routedef search_results(state: State, size: str) -> Page:  state.selected_size = size

Here's why you don't need a separate after_size route:

1. select_size displays the size options.
2. The user selects a size and clicks Continue.
3. search_results receives the selected size.
4. search_results saves it in state.selected_size and filters the birds.

When the user clicks Change size from refine_search, they can go straight back to select_size. Their previous size stays saved until they submit a new one.

So your current approach is correct!

should the program return a bird even if the user inputs are not an exact match

Yes! I would recommend that your bird search engine return birds even when they aren't an exact match. This makes your program much more useful because someone identifying a bird might not remember every characteristic accurately.

For your project, I'd recommend using a **matching system with progressively relaxed criteria**.

**How I would make it work**

For example, suppose a user selects:

- Colors: Blue and White
- Habitat: Wetlands
- Size: Medium

Imagine no bird matches all three characteristics.

Instead of displaying no results, your program could:

1. First, show birds matching all three characteristics.
2. If there are no matches, show birds matching two of the three characteristics.
3. If there are still no matches, show birds matching at least one characteristic.
4. If nothing matches, suggest changing the search.

**An even better approach: matching scores**

You could assign points to each bird based on how many characteristics match.

| **Characteristic** | **Points** |
|---|---|
| Matches selected colors | 1 |
| Matches selected habitat | 1 |
| Matches selected size | 1 |

Then sort birds from highest score to lowest score.

For example:

| **Bird** | **Color** | **Habitat** | **Size** | **Score** |
|---|---|---|---|---|
| Blue Jay | Yes | No | Yes | 2 |
| Belted Kingfisher | Yes | Yes | No | 2 |
| American Robin | No | Yes | Yes | 2 |
| Great Blue Heron | Yes | Yes | Yes | 3 |

*These are illustrative matches.*

**My recommendation for your CS1 project**

I would start with the scoring system, but keep it simple:

- Use a counter variable for each bird.

- Add 1 whenever a characteristic matches.
- Store the bird and its score.
- Display the highest-scoring birds first.

You don't need advanced Python features to do this. Nested loops and conditionals will work perfectly.

One important detail: if the user selects multiple colors, I would count that category as a match if **at least one** selected color matches the bird, rather than requiring every selected color to match.

This would be a nice improvement to your bird search engine because it makes the results more forgiving of imperfect observations.

This might also interest you.

how would i implement this to my coe

Worked for 7s

I would replace your current search_results function with a scoring system. We can do this using the loops and conditionals you've already been using, without needing any().

The idea is:

- Each bird starts with a score of 0.
- Matching a selected color earns 1 point.
- Matching a selected habitat earns 1 point.
- Matching the selected size earns 1 point.
- Your program displays the birds with the highest scores, even if they aren't perfect matches.

**1. Replace your** search_results **function**

```
@route
def search_results(state: State, size: str) -> Page:
    state.selected_size = size


    # Store each bird with its matching score
```

```
bird_scores = []

highest_score = -1

for bird in BIRDS:
  score = 0

  # Check if at least one selected color matches
  for color in state.selected_colors:
    if color in bird.main_colors:
      score = 1
      break

  # Check if at least one selected habitat matches
  for habitat in state.selected_habitats:
    if habitat in bird.habitat:
      score += 1
      break

  # Check if the size matches
  if state.selected_size == bird.size:
    score += 1

  # Save the bird and its score
  bird_scores.append((bird, score))
```

```
    if score > highest_score:
        highest_score = score

# Keep only birds with the highest score
state.matching_birds = []

for bird, score in bird_scores:
    if score == highest_score:
        state.matching_birds.append(bird)

# Display results
content = [
    Header("Your Bird Search Results", 1),
    Text("Here are the birds that best match your selections:")
]

if state.matching_birds == []:
    content.append(Text("Sorry! No birds were found."))
else:
    for bird in state.matching_birds:
        content.append(Header(bird.name, 2))
        content.append(Text("Colors: " + ", ".join(bird.main_colors)))
        content.append(Text("Habitats: " + ", ".join(bird.habitat)))
        content.append(Text("Size: " + bird.size))

content.append(Button("Refine Search", "/refine_search"))
```

```
    content.append(Button("Start Over", "/return_to_start"))


    return Page(state, content)
```

**2. Understand how the scoring works**

Suppose someone searches for:

- Blue
- Wetlands
- Medium

Imagine these are the birds in your database:

| **Bird** | **Color match** | **Habitat match** | **Size match** | **Score** |
|---|---|---|---|---|
| Blue Jay | Yes | No | Yes | 2 |
| Great Blue Heron | Yes | Yes | No | 2 |
| American Robin | No | No | Yes | 1 |
| Belted Kingfisher | Yes | Yes | Yes | 3 |

The Belted Kingfisher would appear first (it has the highest score).

If there were no birds with a score of 3, the program would show the birds with a score of 2 instead.

**3. Two important things to understand**

**Why do we use** break**?**

If a user selects Blue and White, and a bird matches both, we only want to award one point for the color category. break stops the loop once a match is found.

**Why do we use** bird_scores**?**

We need to remember each bird's score so we can find the highest score after checking all the birds. We store each bird and its score together as a tuple:

(bird, score)

One limitation of this version: it only displays birds tied for the highest score, rather than showing every bird in descending score order. I think that's a good first version for your project because it keeps the results manageable.

**One more detail:** If the user leaves a category blank, that category earns no points and doesn't count against a bird.

this unit test is failing: assert_equal(state4.matching_birds, BIRDS)

I think the problem is in how your scoring system handles an empty search.

In your scoring code, every bird gets points for matching selected characteristics. But when the user selects **no colors, no habitats, and no size**, every bird gets a score of 0.

If you added the check I suggested earlier:

```
if highest_score == 0:
    state.matching_birds = []
```

then your test will fail because it expects all birds to be returned.

**Fix: Handle an empty search separately**

In your search_results function, add this immediately after saving the size:

```
state.selected_size = size


if (state.selected_colors == [] and
    state.selected_habitats == [] and
    size == ""):
    state.matching_birds = BIRDS
```

Then, put your scoring algorithm inside an else:

```
else:
    # Your existing scoring algorithm goes here
```

This gives you two different behaviors:

- **No selections:** Return all birds.

- **Selections made, but no birds match:** Return an empty list.

Your original test should then pass:

state4 = State([], [], "", BIRDS)

result4 = search_results(state4, "")

assert_equal(state4.matching_birds, BIRDS)

The distinction is important: an empty search means the user hasn't narrowed anything down, whereas a search with selections that match no birds should return no results.

my website is always returning no matches no matter what i click

I suspect the problem is either that your selected values aren't matching the bird data exactly, or that there's an issue with how your scoring algorithm is saving the results.

Let's check a few things.

**1. Check your habitat names**

In your original select_habitat function, you had:

RelatedCheckBox("habitats", "Forest")

But your bird data may use:

["Forests", "Woodlands", "Urban"]

Those won't match because "Forest" and "Forests" are different strings.

Make sure your habitat options match your BIRDS data exactly. I recommend using "Forests" consistently.

**2. Check your color names**

Your color options and bird data must also match exactly:

- "Blue" is different from "blue".
- "Gray" is different from "Grey".

Python string comparisons are case-sensitive.

**3. Check your scoring algorithm**

Make sure your score is being increased when a match is found:

if color in bird.main_colors:

  score += 1

And similarly for habitats:

if habitat in bird.habitat:

  score += 1

For size:

if size == bird.size:

  score += 1

**4. Let's find the actual problem**

Could you paste your **current complete** search_results **function**, including the scoring algorithm and the code that displays the results?

I especially want to check where you:

- Calculate highest_score.
- Add birds to bird_scores.
- Update state.matching_birds.

I can then identify exactly why you're getting no matches instead of having you change code that might already be correct.